

Choix du bon modèle

1) Tester nos trois modèles avec les deux approches (SMOTE et Class_weight)

Class_weight

- **LR :**
 - AUC-ROC : 0.8222
 - F1-score : 0.50
 - Recall : 0.48
 - Précision : 0.5217
- **RF :**
 - AUC-ROC : 0.8422
 - F1-score : 0.2069
 - Recall : 0.12
 - Précision : 0.75
- **XGB :**
 - AUC-ROC : 0.7797
 - F1-score : 0.4878
 - Recall : 0.40
 - Précision : 0.625

SMOTE

- **LR :**
 - AUC-ROC : 0.7785
 - F1-score : 0.4348
 - Recall : 0.40
 - Précision : 0.4762
- **RF :**

- AUC-ROC : 0.8175
- F1-score : 0.2857
- Recall : 0.20
- Précision : 0.50
- **XGB :**
 - AUC-ROC : 0.7508
 - F1-score : 0.2857
 - Recall : 0.20
 - Précision : 0.50

2) Optimisation du modèle baseline avec GridSearchCV

Recherche des meilleures combinaisons d'hyperparamètres :
81 combinaisons × 5 folds = 405 fits.

Meilleurs hyperparamètres :

- learning_rate : 0.05
- max_depth : 4
- n_estimators : 100
- scale_pos_weight : 1

Meilleur AUC-ROC (CV 5-fold) : 0.8556

Résultat sur test

- **XGB GridSearchCV :**
 - AUC-ROC : 0.8406
 - F1-score : 0.2424
 - Recall : 0.16
 - Précision : 0.50

Analyse des gains :

Métrique	Avant	Après	Delta
AUC-ROC	0.7797	0.8406	+0.0609
F1-score	0.4878	0.2424	-0.2454
Recall	0.40	0.16	-0.24
Précision	0.625	0.50	-0.125

b) Réajustement GridSearch

Le GridSearch précédent (scoring = 'roc_auc') a retourné scale_pos_weight = 1, produisant un modèle biaisé vers la classe majoritaire (Recall = 0.16).

En recentrant sur scoring = 'f1', on maximise directement l'équilibre Précision/Rappel sur la classe churn.

Meilleurs hyperparamètres :

- learning_rate : 0.2
- max_depth : 5
- n_estimators : 100
- scale_pos_weight : 1

Meilleur F1 (CV 5-fold) : 0.6529

Résultats :

- AUC-ROC : 0.7982
- F1-score : 0.375
- Recall : 0.24
- Précision : 0.8571

Toujours insuffisant → passage à Optuna.

C) Optuna

Hyperparamètres optimaux (Optuna v1, 50 essais) :

- n_estimators : 153
- max_depth : 6
- learning_rate : 0.1379
- scale_pos_weight : 3.5322

- min_child_weight : 1
- subsample : 0.6362
- colsample_bytree : 0.7639

Résultats test (seuil 0.50) :

- AUC-ROC : 0.8437
- F1-score : 0.55
- Recall : 0.44
- Précision : 0.7333

Comparaison GridSearch vs Optuna :

Métrique	GridSearch	Optuna v1
AUC-ROC (test)	0.8406	0.8437
F1 (seuil optimal)	0.6809	0.6939
Recall (seuil optimal)	0.64	0.68
Seuil retenu	0.23	0.29
Essais effectués	81 (fixe)	50 (adaptatif)
Espace exploré	discret	continu

C1) Optuna v2 (régularisation + colsample_bylevel)

Hyperparamètres choisis :

- n_estimators : 260
- max_depth : 6
- learning_rate : 0.0268
- scale_pos_weight : 1.5240
- min_child_weight : 4
- subsample : 0.8844
- colsample_bytree : 0.6873
- reg_alpha : 0.8153
- reg_lambda : 2.4416
- gamma : 0.3379

- colsample_bylevel : 0.6073

Résultats test (seuil 0.31) :

- AUC-ROC : 0.8622
- F1 : 0.7059
- Recall : 0.72
- Précision : 0.6923

C2) Optuna v3 (150 essais)

Hyperparamètres :

- n_estimators : 188
- max_depth : 6
- learning_rate : 0.0393
- scale_pos_weight : 1.1058
- min_child_weight : 3
- subsample : 0.8377
- colsample_bytree : 0.6547
- reg_alpha : 0.0795
- reg_lambda : 2.1903
- gamma : 0.1779
- colsample_bylevel : 0.7378

Résultats test :

- AUC-ROC : 0.8646
- F1 : 0.717
- Recall : 0.76
- Précision : 0.6786

C3) Optuna v4 (300 essais)

Hyperparamètres :

- n_estimators : 327
- max_depth : 6
- learning_rate : 0.0146
- scale_pos_weight : 1.3122
- min_child_weight : 4
- subsample : 0.8177
- colsample_bytree : 0.6582
- reg_alpha : 0.5493
- reg_lambda : 0.5035
- gamma : 0.4108
- colsample_bylevel : 0.8634

Résultats test (seuil 0.25) :

- AUC-ROC : 0.8671
- F1 : 0.6909
- Recall : 0.76
- Précision : 0.6333

→ Surapprentissage détecté (F1 baisse malgré AUC en hausse).

C5) Voting Ensemble (LR + RF + XGB optimisé)

Probabilité finale = $(1 \times \text{LR} + 1 \times \text{RF} + 2 \times \text{XGB}) / 4$

Résultats test (seuil 0.25) :

- AUC-ROC : 0.8646
- F1 : 0.7097
- Recall : 0.88
- Précision : 0.5946

Comparaison Optuna v3 vs Ensemble :

Métrique	Optuna v3	Ensemble	Delta
AUC-ROC	0.8646	0.8646	0.0000

Métrique	Optuna v3	Ensemble	Delta
F1	0.717	0.7097	-0.0073
Recall	0.76	0.88	+0.12
Précision	0.6786	0.5946	-0.0840

→ Choix final : **Ensemble**,

Sérialisation via Joblib

Nous avons procédé à la sérialisation du modèle retenu à l'aide de **joblib**. Joblib est optimisé pour les objets `numpy` et `sklearn` : il offre une compression native des tableaux `numpy`, plus rapide et plus stable entre différentes versions de Python.

- Le fichier `.pkl` est ensuite chargé dans **Flask** via `joblib.load()`.
- Nous avons également sérialisé le **seuil optimal (0.21)** dans un fichier séparé, afin que Flask puisse le charger dynamiquement sans le coder en dur dans l'endpoint.

SHAP

1) Problème avec VotingClassifier

Un problème se pose : `shap.TreeExplainer` ne fonctionne pas avec un **VotingClassifier** car il ne sait pas décomposer un modèle hétérogène.

Solution retenue : utiliser le composant **XGBoost** de l'ensemble pour les explications SHAP, et l'ensemble complet pour les prédictions.

- Premièrement, XGBoost a un poids de 2 dans l'ensemble (LR = 1, RF = 1, XGB = 2), soit 50 % de la décision finale.
- Deuxièmement, SHAP dispose d'un explainer spécialisé pour les arbres de décision (`TreeExplainer`), exact et rapide. Pour LR ou VotingClassifier, il faudrait `KernelExplainer`, beaucoup plus lent et approximatif.
- Troisièmement, l'AUC du composant XGBoost seul est de 0.8646 — identique à l'ensemble. Les explications SHAP restent donc représentatives de la décision finale.

2) Calcul des SHAP values

Nous avons choisi le composant XGBoost de l'ensemble avec **TreeExplainer**, spécialisé pour les modèles à base d'arbres.

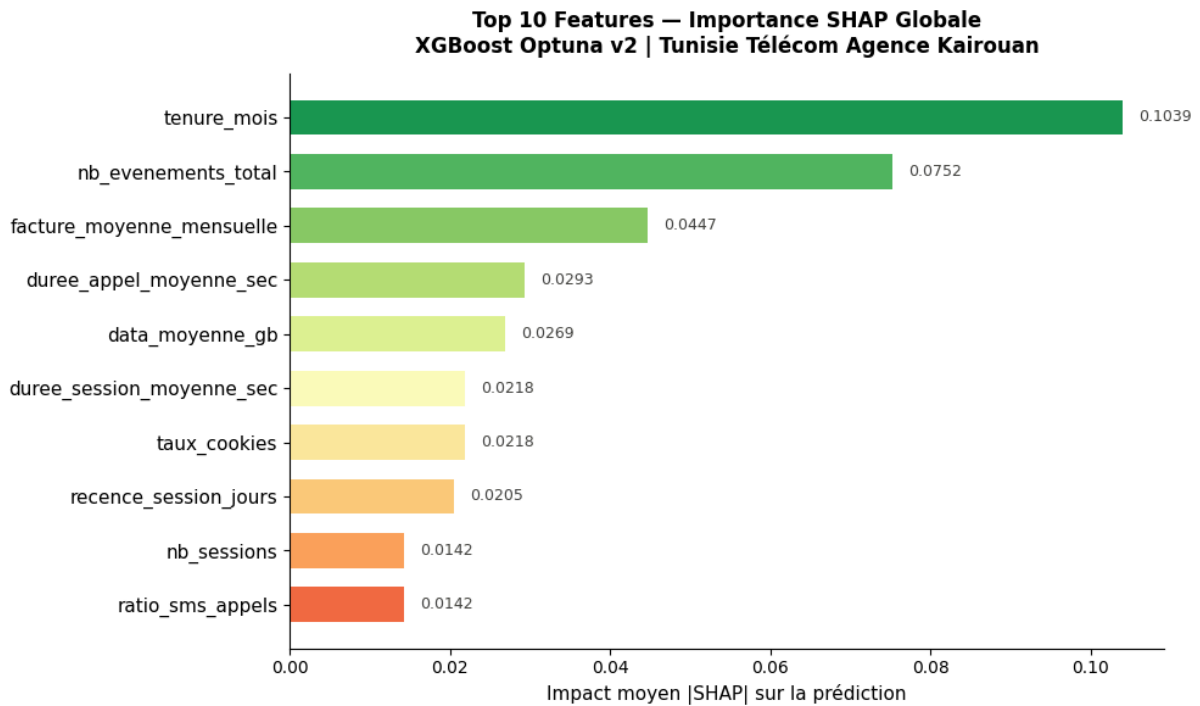
3) Vérification anti-leakage

- recence_cdr_jours : ✓ Absent
- churn : ✓ Absent
- statut_actif : ✓ Absent

4) Top-10 Features (importance SHAP)

Rang	Feature	Impact moyen (SHAP)
1	tenure_mois	0.1039
2	nb_evenements_total	0.0752
3	facture_moyenne_mensuelle	0.0447
4	duree_appel_moyenne_sec	0.0293
5	data_moyenne_gb	0.0269
6	duree_session_moyenne_sec	0.0218
7	taux_cookies	0.0218
8	recence_session_jours	0.0205
9	nb_sessions	0.0142
10	ratio_sms_appels	0.0142

Features peu influentes (rang 11–31) : zone_reseau_principale, qualite_signal_dominante, moyen_paiement, plan_tarifaire, satisfaction_client, score_frustration, etc.

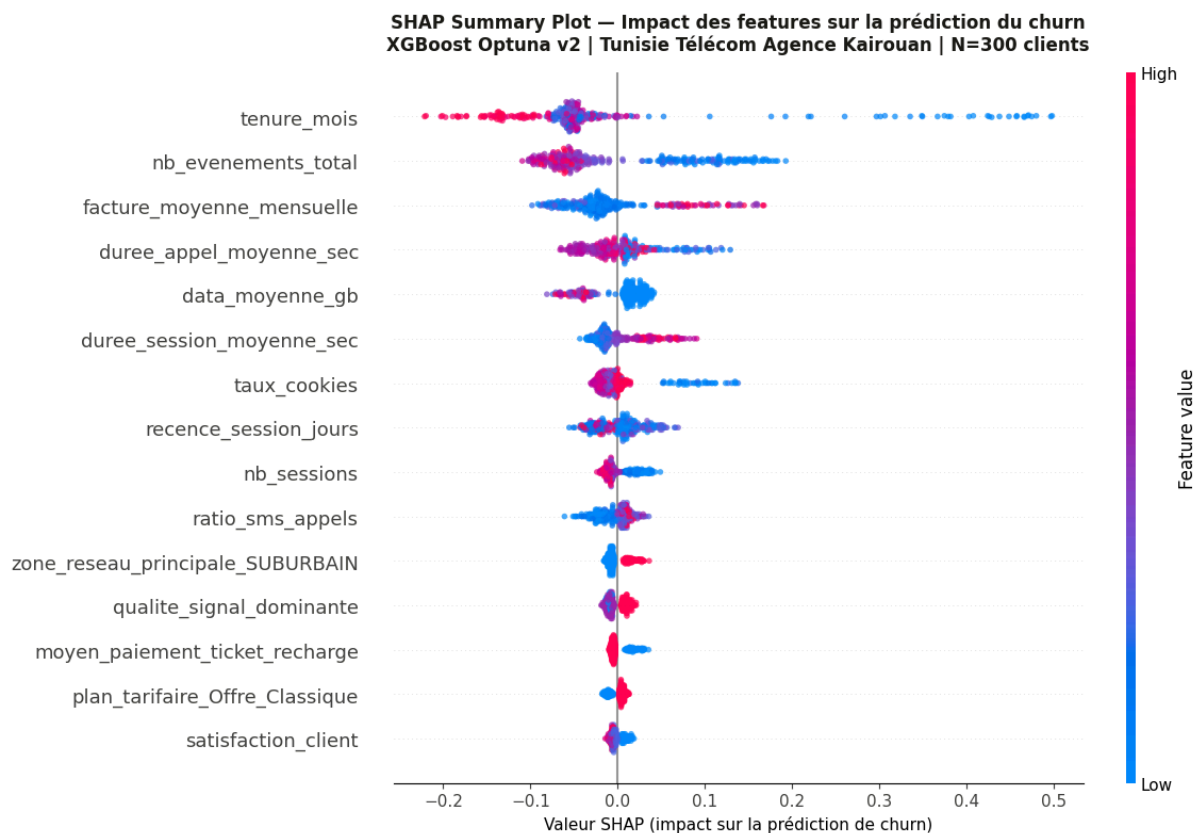


5) Summary Plot Beeswarm

Le **Summary Plot beeswarm** est le graphique SHAP de référence dans la littérature. Chaque point représente un client :

- Position horizontale : force de la contribution SHAP (droite = pousse vers churn, gauche = pousse vers non-churn).
- Couleur : valeur de la feature (rouge = valeur haute, bleu = valeur basse).
- Axe vertical : features classées par importance décroissante.

Il permet de lire simultanément l'importance et le sens de l'effet.



6) Interprétation métier des top features SHAP

Pour chaque feature du top 10, nous avons analysé :

1. Le sens de l'effet (valeur haute → churn ou non-churn).
2. L'interprétation métier pour l'agence de Kairouan.

Exemple : une **facture moyenne mensuelle élevée** peut indiquer un client premium, mais aussi une sensibilité accrue au prix → risque de churn si la valeur perçue baisse.

7) SHAP individuel

Enfin, nous avons procédé à :

- La **prédiction** via l'ensemble complet.
- L'**explication SHAP** via le composant XGBoost.